

Meridian — How the merging works

This is the written version of the algorithm in `packages/core`. It is the document to read before changing anything in `text.js`.

Written: 17 September 2026
Covers: weeks 1 to 9 of the work plan

1. The one rule everything follows

> The result must depend only on **which** changes have arrived, never on **when** > they arrived or **in what order**.

Every decision below comes from that rule. Any code that breaks it is a bug, even if the tests happen to pass.

This is why the design never uses:

- the computer's clock — two machines disagree, so the same changes would merge differently on each one;
- arrival order — the network is allowed to deliver in any order it likes;
- "who connected first" — that would need a server to decide, and there is no server here.

2. Naming changes

Every change gets an id: `"<counter>@<site>"`, for example `"12@a3f9c1d8"`.

- **site** is a random id made once per device.
- **counter** goes up by one for every change that device makes.

Two ids are compared by counter first, then by site. The site part is only a tie-break. It does not matter which device wins a tie — only that every device picks the same winner.

When a device loads a saved document, it moves its counter past every id **of its own** that it has seen. Without that, it would hand out an id it had already used, and two different changes would share one name.

Two counters, not one

Every change also carries a second number, called its stamp. A change therefore has a **name** and a **position in an order**, and these need different rules:

	NAME (<code>COUNTER</code>)	STAMP (<code>LAMPORT</code>)
Must be gapless per device	Yes	No
Moves for other devices' changes	No	Yes
Used for	Syncing, deduplication	Ordering

One counter cannot do both, and using one for both was a real bug in this project. The story is worth keeping:

Syncing sends a summary of the form *"I hold everything from device X up to number N"*. That only means something if each device's own numbering is unbroken. But ordering needs the opposite: a letter typed between two others must beat the letter on its right, which means taking account of numbers from other devices.

Doing both with one counter put holes in each device's own sequence — its next change might be number 9, with 1 to 8 never existing. The summary then could not move past the hole, so **the device reported holding none of its own changes**. Peers concluded it was permanently behind and greeted it forever, while the text itself was perfectly in sync. It took a partition test running over several rounds to see it at all.

Code: `src/id.js`.

3. Text as a tree, not a string

The document is stored as a tree of single letters. Each letter holds:

FIELD	MEANING
<code>id</code>	Its permanent name
<code>parent</code>	The id of the letter it was typed after, or <code>null</code> for the start
<code>value</code>	One character
<code>deleted</code>	Whether it is hidden

The text you read is the tree walked depth-first: a letter, then everything typed after that letter, then the next sibling.

The tie-break

Several letters can share a parent. That is what a conflict is. They are sorted by *stamp*, newest first, with the device id breaking an exact tie.

The stamp, not the name. A letter typed between two others was written by someone who had *seen* the letter on its right, so its stamp beats it and it lands in the right place. Sorting by name instead let a device with a low counter push its letter to the wrong side of a letter it was deliberately typed in front of.

Because the tree is built only from `id` and `parent` — both of which never change — and the sort is fixed, every device builds the same tree from the same set of changes. The same tree read the same way gives the same text. No server decides anything.

Why typed words stay whole

When you type `hello`, each letter's parent is the letter before it. The word is a **chain**, not five siblings.

If someone else types `world` in the same spot, their chain hangs off the same parent as your `h`. The sort puts one whole chain before the other:

`helloworld` or `worldhello`

never `hweolrlld`. This is the main reason the tree is shaped this way.

Code: `src/text.js`.

4. Deleting

A deleted letter is marked hidden and kept. It is called a tombstone.

It cannot actually be removed, because another device may still send a change that refers to it. If it were gone, that change would have nowhere to attach.

Two things follow:

- **Position mapping matters.** Hidden letters must not be counted when working out where a new letter goes. Getting this wrong looks correct until something is deleted. There is a test for exactly this.
 - **Tombstones pile up.** Removing them safely needs proof that every device has moved past them. That is snapshot work, and it is not built yet.
-

5. Changes that arrive early

A change may arrive before the change it depends on. Two cases:

CASE	WHAT HAPPENS
An insert whose parent is unknown	Held in <code>waiting</code> , keyed by the parent it needs. Applied the moment that parent arrives.
A delete for an unknown letter	Its target is remembered in <code>waitingDeletes</code> . When the letter arrives, it is born hidden.

Both are required, not defensive extras: the convergence test delivers changes in random order, so this path runs constantly.

`text.settled()` reports whether anything is still held back. A replica that has received every change and is not settled means something referred to a letter that was never sent — a real bug, and the tests assert on it.

6. Applying a change twice

Every apply path is written so that applying the same change again does nothing:

- insert — ignored if the id is already known;
- delete — already hidden, so nothing changes;
- field — the same id cannot beat itself;
- tag — adding an id to a set twice is one entry.

This matters because a peer is allowed to resend anything at any time, and after a reconnect it usually will.

7. The other two data types

Fields (`src/map.js`) — one value per name, such as the title. Highest id wins. Losing one of two titles is correct; there is no sensible way to merge them. What matters is that everyone loses the same one.

Tags (`src/set.js`) — a set where adding beats removing. Each add carries its own tag. A remove can only remove tags it could actually see, so a tag added at the same moment as a remove survives.

Add-wins is the safer default for shared documents: losing a tag someone just added is confusing, while seeing one you thought you removed is merely annoying.

8. Syncing

Sending the whole document on every meeting would work and would be unusable on a slow connection. Instead each device keeps a small summary, called a version vector: for each device, the highest counter it holds **with no gaps**.

```
{ "a3f9c1d8": 42, "7b2e0c11": 17 }
```

Two peers swap summaries and each replies with only what the other lacks.

The "no gaps" part is important. If a device holds change 1 and change 3 but not 2, its summary says 1. Claiming 3 would make a peer skip change 2 forever.

A change already held may occasionally be re-sent when it sits past a gap in the other side's history. That is a little extra traffic in exchange for a summary that stays tiny, and applying a change twice does nothing.

Code: `src/oplog.js`.

9. Saving to disk

The rule: **a change is written to this device before it is sent anywhere**. A device that loses local work is worse than one that cannot sync, because the user can see it happen.

What a store has to do

METHOD	JOB
<code>site()</code>	This device's id, made once and kept forever
<code>append(ops)</code>	Add changes. Must ignore ones it already has
<code>read()</code>	Everything needed to rebuild the document
<code>replace(snapshot)</code>	Save the state and delete the changes, as one step

Three adapters exist, and all three pass the same test kit (`@meridian/core/testing`). One shared kit rather than one suite per adapter, so "correct" means one thing instead of three slightly different things.

ADAPTER	WHERE	BUILT ON
MemoryStore	Tests, and the reference to copy	Nothing
SqlStore	Desktop and Node	node:sqlite, or better-sqlite3
IdbStore	Browser	IndexedDB

SqlStore takes a database object and only calls `exec` and `prepare`. Both drivers provide exactly that, so the same file works with either. This matters because Electron ships its own build of Node, and whichever driver is available there, the adapter does not change.

Writes are queued

Changes arrive from a plain function call that cannot wait. Writing to disk can. So Persistence keeps a queue and writes one batch at a time, in order.

Without the queue, two edits in the same millisecond race, and on some stores the second is lost. There is a test with a deliberately slow store that fails if the queue is removed.

`flush()` waits for the queue and **raises the first write error**. A failing disk must not be swallowed: the app has to be able to tell the user their work is not being saved.

Why `replace` must be one transaction

Saving state and deleting changes are one step on purpose:

- crash *after* writing state but *before* deleting changes — harmless, the changes are simply replayed;
- crash *after* deleting changes but *before* writing state — **the document is gone**.

10. Compacting, and the trap inside it

Tombstones and old changes pile up forever, so eventually the stored changes are replaced by a snapshot of the state.

This introduces a real danger. Once the changes are gone, this device can no longer hand them out one by one. A peer that is behind the trim point would ask for them, receive nothing, and **quietly stay wrong forever**. Sync would report success the whole time.

So the change log records what it has thrown away:

- `trimmed` — the version vector whose result we hold but whose changes we cannot produce;
- `tooFarBehind(theirs)` — true when a peer needs something inside that range;
- when it is true, `syncTo` sends **whole state** instead of changes.

Merging whole state

The tempting shortcut is to overwrite the local document with the incoming state. That would destroy any local work the sender had not seen. So it is a merge, not a load:

PART	HOW TWO STATES MERGE
Text	Union of letters by id. A deletion on either side wins
Fields	Per name, the higher id wins
Tags	Union of added tags, union of removed tags

Every part only ever grows, and deleting never reverses, so merging states is safe to repeat and order does not matter — the same property the individual changes have. A snapshot that predates a deletion cannot bring the letter back, and there is a test for exactly that.

This is also how a brand new device can join a long-lived document without replaying its entire history.

11. The network

Nothing in `packages/sync` mentions WebRTC or WebSocket. A connection is just something that carries strings:

```
js { send, onMessage, isOpen, onClose?, close? }
```

That is what makes the protocol testable over a fake network that drops, duplicates and reorders messages, which is where its real bugs were found. The one file that does know about WebRTC — `src/webrtc.js` — only opens a pipe and hands it over.

Four messages

MESSAGE	MEANING
<code>hello</code>	Here is everything I have
<code>ops</code>	Here are changes you are missing
<code>want</code>	I am too far behind for changes, send the whole document
<code>state</code>	Here is the whole document

Every hello is answered with a hello, marked as an answer so the exchange stops after one round trip. That reply is not a nicety; without it the protocol does not work. A peer cannot know what to send until it knows what the other side has, and a peer holding nothing of its own believes it is up to date and so never speaks first. Both sides then sit silently, each waiting for the other, while the documents differ.

There is no message queue

A dropped message, a connection that died mid-sentence, and a laptop shut for a week all have the same cure: say hello again. From one hello each side works out exactly what the other lacks, so anything lost is re-derived rather than retransmitted. Sends are allowed to fail silently for that reason.

Repair is a tick, not an event

A handshake cannot survive being lost. On a link dropping a third of its messages the hello itself goes missing, and then neither side says anything more.

So `Network.tick()` greets any peer that has not confirmed it holds everything we hold. The app calls it on a timer, every few seconds. Two details make it work:

- **A hello doubles as the acknowledgement.** Nothing says "received"; the only evidence a peer got our changes is its next hello reporting a version that covers them.
- **A tick sends changes too, not just a greeting.** Greeting alone would wait for a reply before sending anything, which is the silent stand-off described above.

Measured: at 30% packet loss, two peers reconcile in **under a dozen tick rounds**, and the test fails rather than looping if repair is ever unbounded.

Relaying, and why it terminates

Changes from one peer are passed to the others, so a group where everyone connects to one person still syncs fully.

This sounds like it should never stop — A tells B, B tells A — and it stops for a precise reason: `Doc.receive` reports only the changes that were **new**, and only those are relayed. The second time a change comes round there is nothing to pass on, so it dies there. The change ids do the work a hop counter would otherwise have to.

Messages from peers are checked

Anything arriving from the network was written by someone else, who may be running an older build, a newer build, or trying to break things. Malformed messages are dropped, not thrown, because a bad message is an expected event on a public network. A batch of changes containing one malformed change is refused whole — accepting the good half would apply a change with no usable id, which could then be applied over and over.

Introductions

Two browsers cannot call each other directly, so `server/signal` relays the connection details they need to find each other. It sees nothing else, stores nothing, and is trusted by neither peer. Once connected, changes travel directly and the server could be switched off without either side noticing.

Rooms are named by a **hash** of the document id, so the server can tell that two people want the same room without learning which document it is.

The newcomer makes the WebRTC offer, because the server tells an arriving peer who is already present. If both offered at once the handshake would collapse — WebRTC calls that a signalling collision.

12. The editor

Two problems stand between a correct merge algorithm and an editor anyone would use. Neither is solved by the algorithm, and both are pure logic, so both are tested in Node rather than discovered by hand in a browser.

Turning a text box into changes

A text box hands over its whole contents after every keystroke. The document needs the opposite: the smallest description of what altered.

`diff()` matches the identical text at the start, then at the end, and whatever is left in the middle is the change.

Replacing the whole document instead would still *look* right on one machine. It would also delete and re-insert every letter, so two people typing in the same paragraph would obliterate each other, every cursor would jump, and the change log would grow without bound. There is a test asserting that adding two letters produces exactly two changes.

One trap, with a test of its own: `"aa"` becoming `"aaa"` matches two characters at the start *and* two at the end of a three-character string. Without a guard the two ranges overlap and the removed count goes negative.

Keeping the cursor still

A cursor cannot be stored as a number. Someone typing earlier in the document shifts every number after it, and the cursor appears to jump — the single most irritating bug in collaborative editors.

So a cursor is stored as **the id of the letter it sits after**. That letter is the same letter no matter what anyone types elsewhere. `anchorAt(index)` and `indexAfter(anchor)` convert between the two.

Two details that are easy to get wrong:

- A **deleted** letter still works as an anchor. If someone else removes the letter your cursor sat after, the cursor belongs where that letter was, not at the start of the document.
- An **unknown** anchor resolves to the end. It means a cursor referring to letters that have not arrived yet; the end is wrong but harmless, while the start would look like the document had scrolled itself.

Presence is not part of the document

Who is here and where their cursor is travels as a fifth message type, `who`, and never touches the document. It is not saved, not merged and not replayed. Storing it in the document would mean a cursor position from last Tuesday syncing forever.

It is capped at 4 KB, because a channel nothing else checks is otherwise a way to push bulk data at everyone in the room.

Two tabs are two devices

The bug that only a browser could have found.

Two tabs of one browser share one IndexedDB. Both opened it, both adopted the stored device id, and so **they became the same device**. Their change ids collided, each discarded the other's edits as duplicates it already had, and the two tabs disagreed permanently while every status indicator said they were in sync.

Two tabs are two replicas and need two identities. The tab's id now lives in `sessionStorage`, which is per tab and survives a reload — exactly the lifetime wanted. They still share a store, which is fine: changes are globally unique and merge.

This also changed how a document is loaded. It is now built by **merging** the saved state rather than loading it, because loading adopts whichever device wrote the snapshot — right for one device, wrong for the second tab,

whose own identity has to survive.

13. How this is tested

TEST	WHAT IT PROVES
<code>text.test.js</code>	The specific cases that are easy to get wrong, including editing after a deletion and receiving changes backwards
<code>types.test.js</code>	Fields, tags, the version vector, and save/load
<code>convergence.test.js</code>	The real proof: hundreds of random runs where replicas edit apart and then receive everything in different random orders
<code>storage.test.js</code>	Reload, crash, write ordering, write failure, compacting, and state merge
<code>storage-sql</code> , <code>storage-idb</code>	The same shared kit, run against SQLite on disk and against IndexedDB
<code>sync/session.test.js</code>	Handshakes, relaying, reconnecting, and refusing malformed messages
<code>sync/unreliable.test.js</code>	Loss, duplication, reordering and partitions, all seeded and replayable
<code>signal/signal.test.js</code>	The introduction service, over real sockets
<code>core/editing.test.js</code>	Diffing and cursor anchoring, including 2000 random diff pairs
<code>sync/presence.test.js</code>	Presence arriving, expiring, and never reaching the document
<code>tools/two-tabs.js</code>	The whole thing, in two real browsers

Every run prints its seed. Replay a failure with:

```
bash SEED=12345 npm test
```

The test suite has teeth

A test that cannot fail is worthless, so each of these was deliberately broken again to check the suite noticed:

BUG REINTRODUCED	RESULT
Sibling sort by arrival order	9 tests fail, including all 5 convergence tests
One-way handshake (no hello reply)	3 sync tests fail
Counters absorbing other devices' numbers	1 test fails
Two tabs sharing one device id	1 test fails, and the browser test disagrees on text

The last one is the interesting result. Only its own regression test caught it, and the network tests did not — because the flaw is symmetric: both sides under-report their versions identically, so the mistakes cancel out and sync looks healthy. A bug that hides from end-to-end tests is exactly the kind worth a unit test of its own.

Worth repeating after any change to `text.js`, `id.js` or `session.js`.

14. What is known to be missing

Written down honestly, because a limitation you know about is a plan and one you have hidden is a trap.

LIMITATION	EFFECT	WHEN
Interleaving	Two people editing <i>inside</i> the same concurrent run can still mix. Convergence is unaffected — everyone sees the same mixed result. Fixing it means a stronger ordering rule, such as Fugue or YATA.	Later
Tombstones are never dropped	Memory grows with every deletion ever made. Compacting trims the change log but keeps hidden letters, because a peer may still refer to them	Needs a way to prove every device has moved past them
Compacting is manual	Nothing decides when to compact yet. It needs a trigger, such as a change count or an age	Week 10
Reading order is rebuilt after each change	Cached, so a burst costs one walk — but it is still $O(\text{document})$ per edit and will need a proper index	Performance week
No encryption yet	Changes travel as plain data	Week 11
The WebRTC transport has no unit tests	<code>src/webrtc.js</code> cannot run in Node. It is now covered end to end by <code>tools/two-tabs.js</code> , which drives two real browsers, but not by the ordinary suite	Stands
Remote cursors are not drawn	Their positions arrive and are held; a plain <code><textarea></code> cannot paint another person's caret. Needs a rendered editor rather than a text box	Later
Plain text only	No formatting, and the field and tag types are not yet used by the interface	Later
No relay fallback	A minority of strict networks — symmetric NAT, some corporate firewalls — cannot connect directly at all. The honest answer is a TURN relay, which is not built	Later

| **IndexedDB is tested against a stand-in** | `fake-indexeddb` exercises the real transaction flow, but not a real browser | Week 9, with the web app |

The first one is the interesting one, and it is a deliberate trade. Convergence was the goal for week 4, and convergence is proven. Better ordering is an improvement on top of something correct, not a missing foundation.